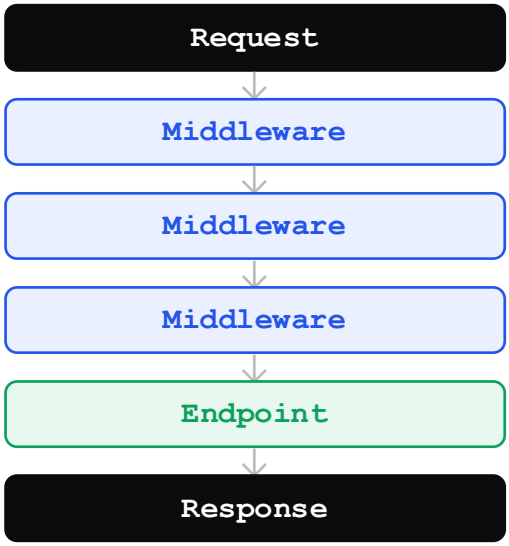
Middleware: what actually happens between Request and Response



Middleware is a component in the ASP.NET Core request pipeline. Every middleware can inspect the request, change it, pass it forward, or stop the pipeline.

The most important concept is next().

Middleware runs on the way IN

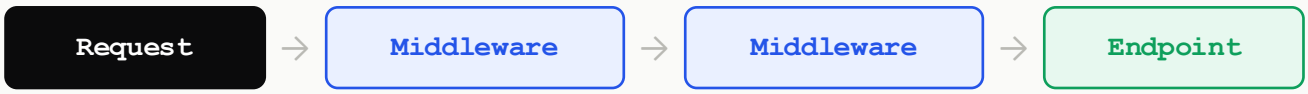
— and can run again on the way OUT.

IN THIS CAROUSEL

Pipeline → next() → Short-circuit → Order → Real examples → Deep concepts

What is Middleware?

Middleware is code that sits between the incoming HTTP request and the final endpoint.

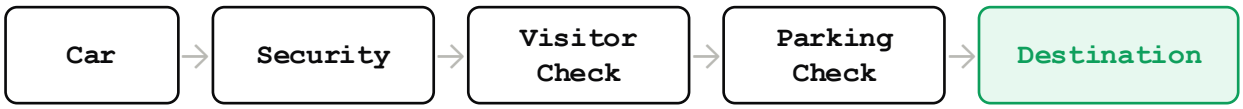


Middleware can:

- Read the request
- Change the request
- Add information
- Check authentication
- Log the request
- Handle exceptions
- Return a response
- Pass the request to the next middleware

A car entering a company

ANALOGY



Think of an HTTP request as a car. Every checkpoint is a middleware.

- Security may check the car
- Visitor verification may check permission
- Another checkpoint may record the entry
- Finally, the car reaches its destination

CAR

HTTP Request

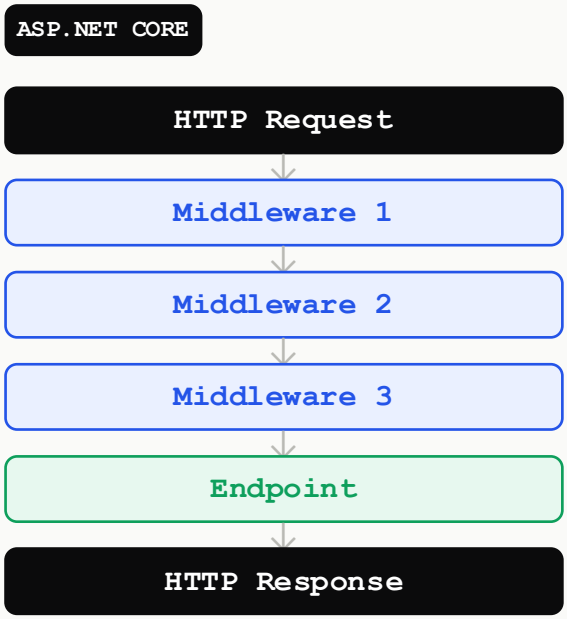
CHECKPOINT

Middleware

DESTINATION

Endpoint

What happens to the request?



The request does not jump directly to the controller. It travels through the configured middleware pipeline.

```
app.UseMiddleware<LoggingMiddleware>();

app.UseMiddleware<AuthMiddleware>();

app.MapControllers();
```

Each line above adds one more checkpoint the request must pass through, in order.

The most important part: next()

```
public async Task InvokeAsync(HttpContext context)
{
    // Before

    await _next(context);

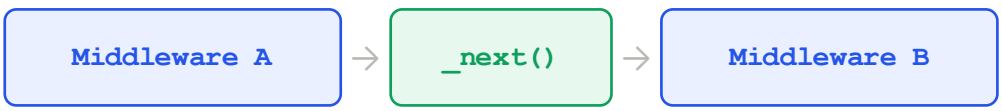
    // After
}
```

_next means: continue to the next middleware.

ANALOGY

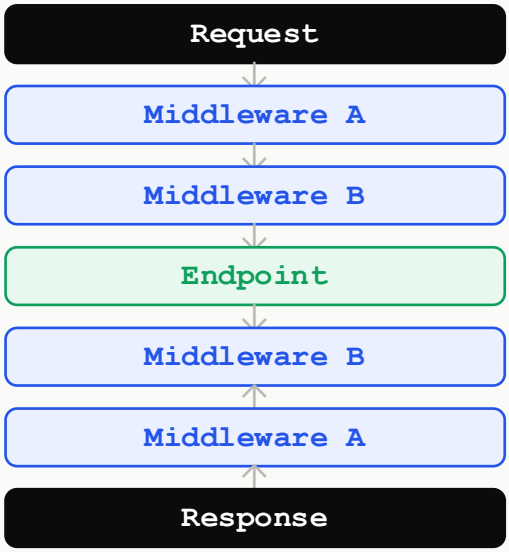
Security checkpoint says: **"You passed my check. Continue to the next checkpoint."**

That is basically what next(context) does.



The checkpoint hands the car forward — it doesn't teleport there on its own.

Middleware works in TWO directions



Middleware can run code BEFORE calling _next() — and code AFTER it completes.

```
Console.WriteLine("Before");

await _next(context);

Console.WriteLine("After");
```

ANALOGY

The checkpoint checks the car when it enters. But it can also do something when the car leaves the checkpoint.

Middleware can STOP the pipeline

```
if (!IsAllowed(context))
{
    context.Response.StatusCode = 403;
    return;
}

await _next(context);
```

Calling _next() is not mandatory. If middleware returns a response without calling it, the request does not continue.

ANALOGY

Security finds a problem. The car is stopped. It never reaches the next checkpoint.



Instead of continuing to Next → Endpoint

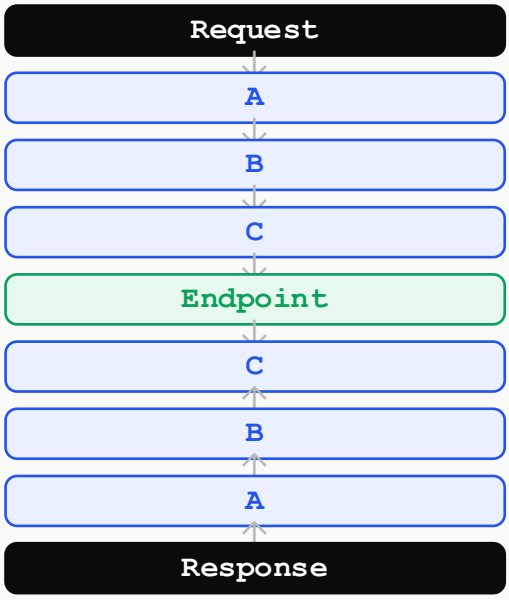
REMEMBER

Short-circuiting is a feature, not a bug — it's how auth and validation protect the rest of the pipeline.

Middleware ORDER matters

```
app.UseA();
app.UseB();
app.UseC();
```

Middleware runs in the exact order it is added to the pipeline.



ANALOGY

You cannot put the final destination before the security checkpoint and expect the same result. Middleware order can change your application's behavior.

A real ASP.NET Core middleware

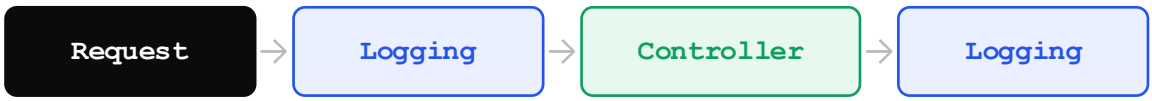
ASP.NET CORE

```
public async Task InvokeAsync(HttpContext context)
{
    Console.WriteLine("Request started");

    await _next(context);

    Console.WriteLine("Request finished");
}
```

```
app.UseMiddleware<RequestLoggingMiddleware>();
app.MapControllers();
```



This is why middleware is useful for logging, exception handling, authentication, security headers, and other cross-cutting work.

PATTERN

Before-and-after logging is the simplest example — the same shape powers timing, auditing, and response-wrapping middleware.

Middleware vs MVC Filters vs Endpoint Filters

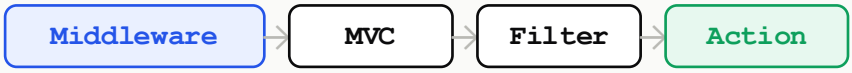
MIDDLEWARE

Works at the HTTP pipeline level.



MVC FILTERS

Work around MVC controller / action execution.



ENDPOINT FILTERS

Work around endpoint execution for supported endpoint types.

Middleware is broader. Filters sit closer to endpoint execution — middleware is not just another type of controller filter.

DON'T THINK

"Middleware is just another type of controller filter."

Endpoint metadata + common mistakes

After routing picks an endpoint, ASP.NET Core can attach metadata to it.



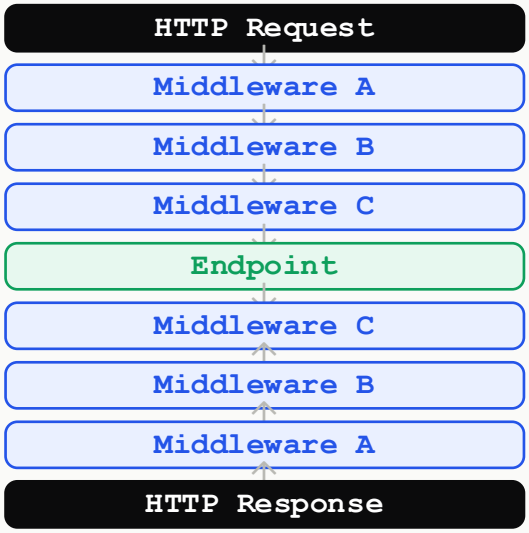

```
var endpoint = context.GetEndpoint();
var metadata = endpoint?.Metadata;
```

This lets middleware decide: does this endpoint need authorization? Does it carry a custom attribute?

COMMON MISTAKES

1. Forgetting _next(context) when the request should continue.
2. Putting middleware in the wrong order.
3. Doing heavy work on every request without thinking about performance.
4. Using middleware when an MVC / endpoint filter is a better fit.
5. Forgetting it can run before AND after _next().

The car has reached its destination



- Middleware is a pipeline
- next() moves the request forward
- No next() can stop the request
- Order matters
- Middleware can work before and after the next component
- Middleware can use endpoint information to make decisions

Now when you see app.Use(...), don't just read it as configuration. Think about the pipeline being built.

What should we deep dive into next: Use, Run, Map, or UseRouting?